

Conceptuales

1. **¿Qué es un objeto en JavaScript?**
Es una colección de pares clave-valor (propiedades y métodos). Es una entidad que agrupa datos y comportamientos relacionados.
2. **¿Qué diferencia hay entre clase y objeto?**
La clase es el "plano" o plantilla que define la estructura (propiedades y métodos), mientras que el objeto es la instancia concreta creada a partir de esa plantilla.
3. **¿Qué es un prototipo?**
Es un objeto del cual otros objetos heredan propiedades y métodos. En JS, cada objeto tiene una propiedad interna `[[Prototype]]` que apunta a su prototipo.
4. **¿Qué hace la palabra `new`?**
Crea un nuevo objeto, vincula su prototipo al de la función constructora, ejecuta dicha función con `this` apuntando al nuevo objeto y lo devuelve.
5. **¿Por qué JS no es realmente orientado a clases?**
Porque utiliza herencia basada en prototipos (*prototypal inheritance*), no en clases. Las `class` en JS son "azúcar sintáctico" sobre el modelo de prototipos existente.

Técnicas

6. **¿Qué diferencia hay entre método dentro del constructor vs. en prototype?**
En el constructor, el método se crea por cada instancia (ocupa más memoria). En `prototype`, el método se define una sola vez y todas las instancias lo comparten por referencia.
7. **¿Qué hace `extends`?**
Establece la cadena de prototipos entre dos clases, permitiendo que la clase hija herede las propiedades y métodos de la clase padre.
8. **¿Para qué sirve `super()`?**
Llama al constructor de la clase padre. Es obligatorio en una clase hija antes de acceder a `this`.
9. **¿Qué es la prototype chain?**
Es el mecanismo por el cual JS busca una propiedad o método: primero en el objeto mismo, luego en su prototipo, luego en el prototipo del prototipo, hasta llegar a `null`.

Pensamiento

10. **¿Cuándo conviene usar POO y cuándo no?**
Conviene cuando tienes entidades complejas con estado compartido y comportamiento repetitivo. No conviene para scripts simples, lógica funcional pura o cuando la herencia añade complejidad innecesaria.
11. **¿Qué ventaja tiene sobre código "normal"?**
Mejora la organización, la reutilización mediante la herencia, el encapsulamiento de datos y facilita el mantenimiento en proyectos grandes.
12. **¿Qué problema resuelve la herencia?**
Evita la duplicación de código al permitir que objetos especializados reutilicen la lógica definida en estructuras más generales.